ENEL 573 – Computer Networks

Chapter 5: Transport Layer

Instructor: Dr. Qiang (John) Ye
Department of ESE
University of Calgary, Canada
Email: qiang.ye@ucalgary.ca

Chapter 5: Transport Layer

Our goals:

- understand principles behind transport layer services:
  - multiplexing/demultiplexing
  - flow control
  - congestion control
- learn about transport layer protocols on the Internet:
  - UDP: connectionless transport
  - TCP: connection-oriented transport
  - TCP congestion control

Chapter 5: Outline

5.1 Transport-layer services

5.2 Multiplexing and demultiplexing

5.3 Connectionless transport: UDP

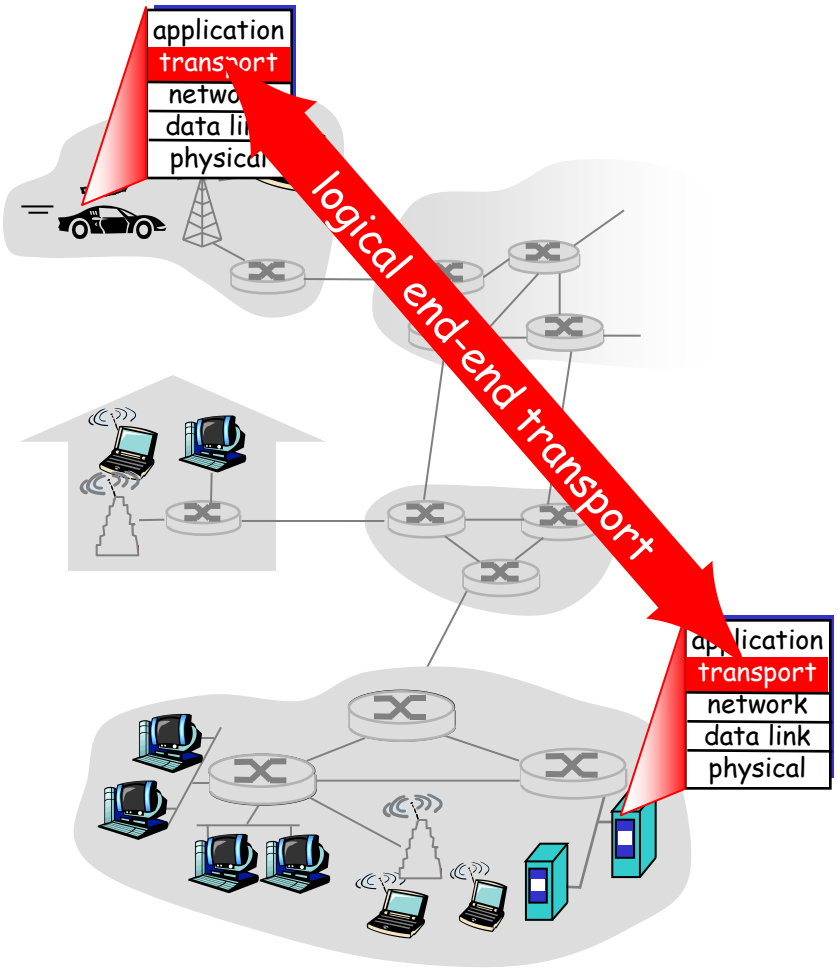
5.4 Connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

5.5 TCP congestion control

Transport services and protocols

- provide *logical communication* between app processes running on different hosts
- transport protocols run on end systems
  - send side: breaks app messages into segments, passes to network layer
  - rcv side: reassembles segments into messages, passes to app layer
- more than one transport protocol available to apps
  - Internet: TCP and UDP



Transport vs. network layer

- *network layer:* logical comm. between hosts
- *transport layer:* logical comm. between processes
  - relies on and enhances network layer services

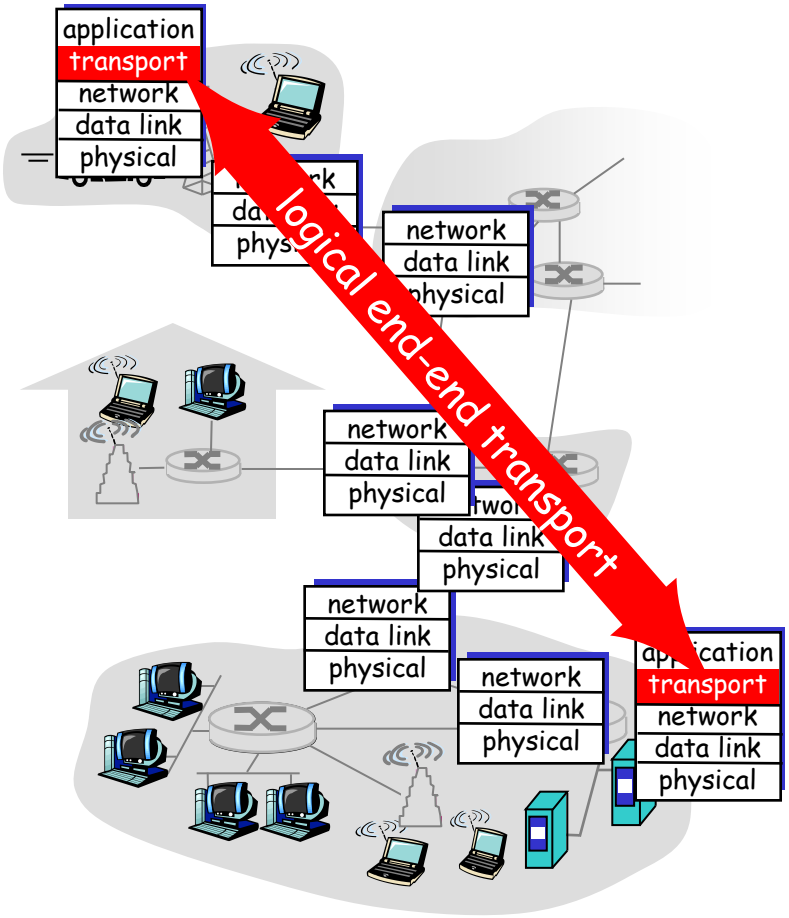
Household analogy:

12 kids sending letters to 12 kids

- processes = kids
- app messages = letters in envelopes
- hosts = houses
- transport protocol = Ann and Bill who demux to in-house siblings
- network-layer protocol = postal service

Internet transport-layer protocols

- reliable, in-order delivery (TCP)
  - congestion control
  - flow control
  - connection setup
- unreliable, unordered delivery: UDP
  - Simple extension of "best-effort" IP
- services not available:
  - delay guarantees
  - bandwidth guarantees



Chapter 5 outline

5.1 Transport-layer services

5.2 Multiplexing and demultiplexing

5.3 Connectionless transport: UDP

5.4 Connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

5.5 TCP congestion control

Multiplexing/Demultiplexing

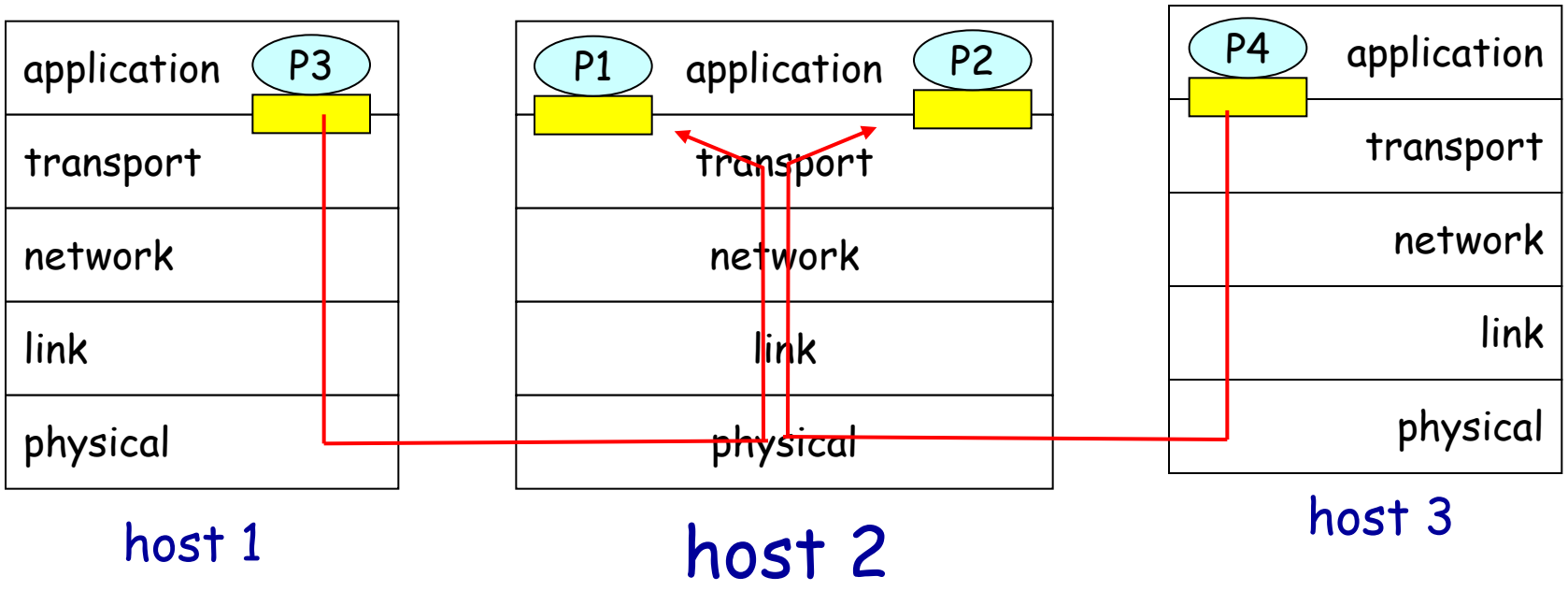
Demultiplexing at rcv host:

delivering received segments to correct socket

Multiplexing at send host:

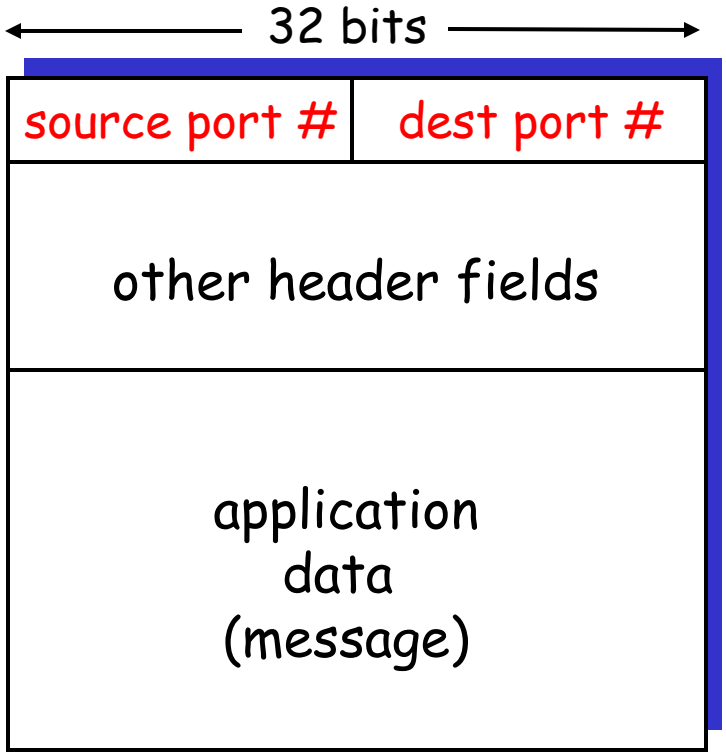
gathering data from multiple sockets, enveloping data with header (later used for demultiplexing)

= socket = process



How demultiplexing works

- host receives IP datagrams
  - each datagram has source IP address, destination IP address
  - each datagram carries 1 transport-layer segment
  - each segment has source, destination port number
- host uses IP addresses & port numbers to direct segment to appropriate socket



TCP/UDP segment format

Connectionless demultiplexing

- *recall:* create sockets with host-local port numbers:

```
DatagramSocket mySocket1 = new
   DatagramSocket(12534);
DatagramSocket mySocket2 = new
   DatagramSocket(12535);
```

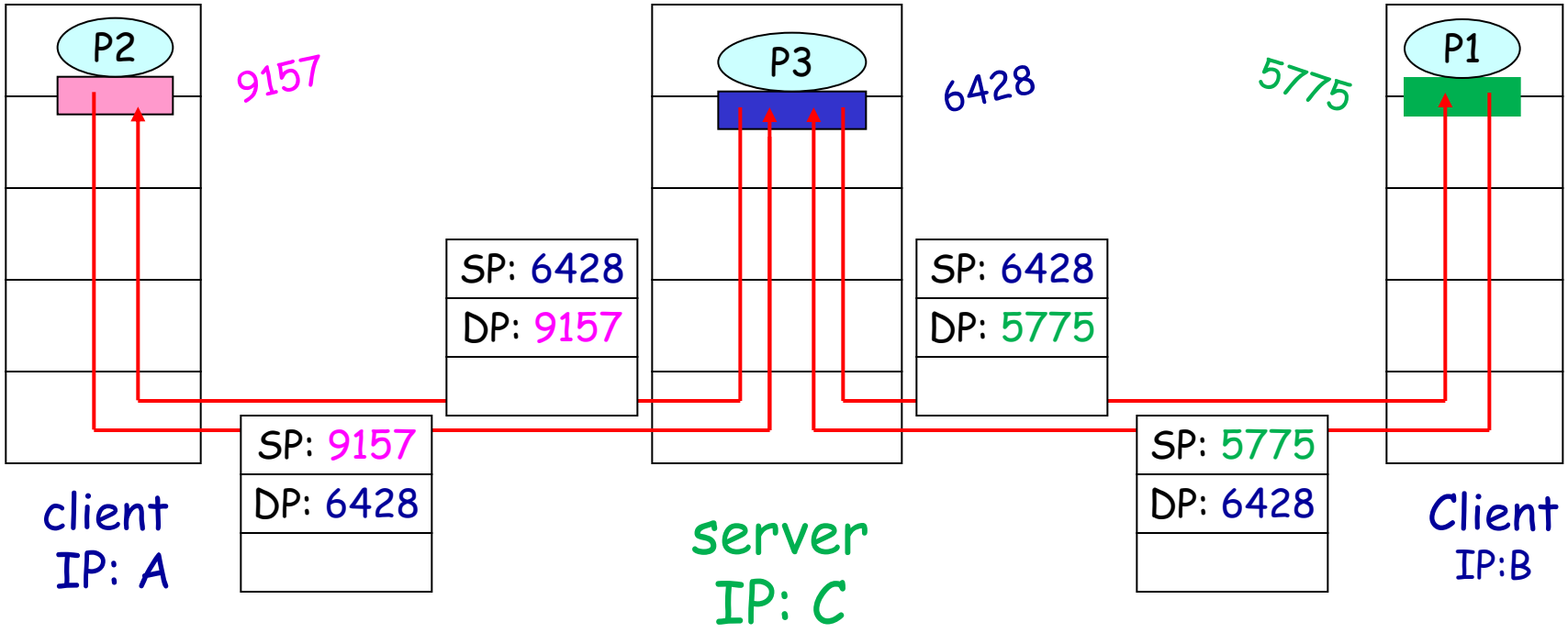
- *recall:* when creating datagram to send into UDP socket, must specify

(dest IP address, dest port number)

- when host receives UDP segment:
  - checks destination port number in segment
  - directs UDP segment to socket with that port number

Connectionless demux (cont)

```
DatagramSocket serverSocket = new DatagramSocket(6428);
```

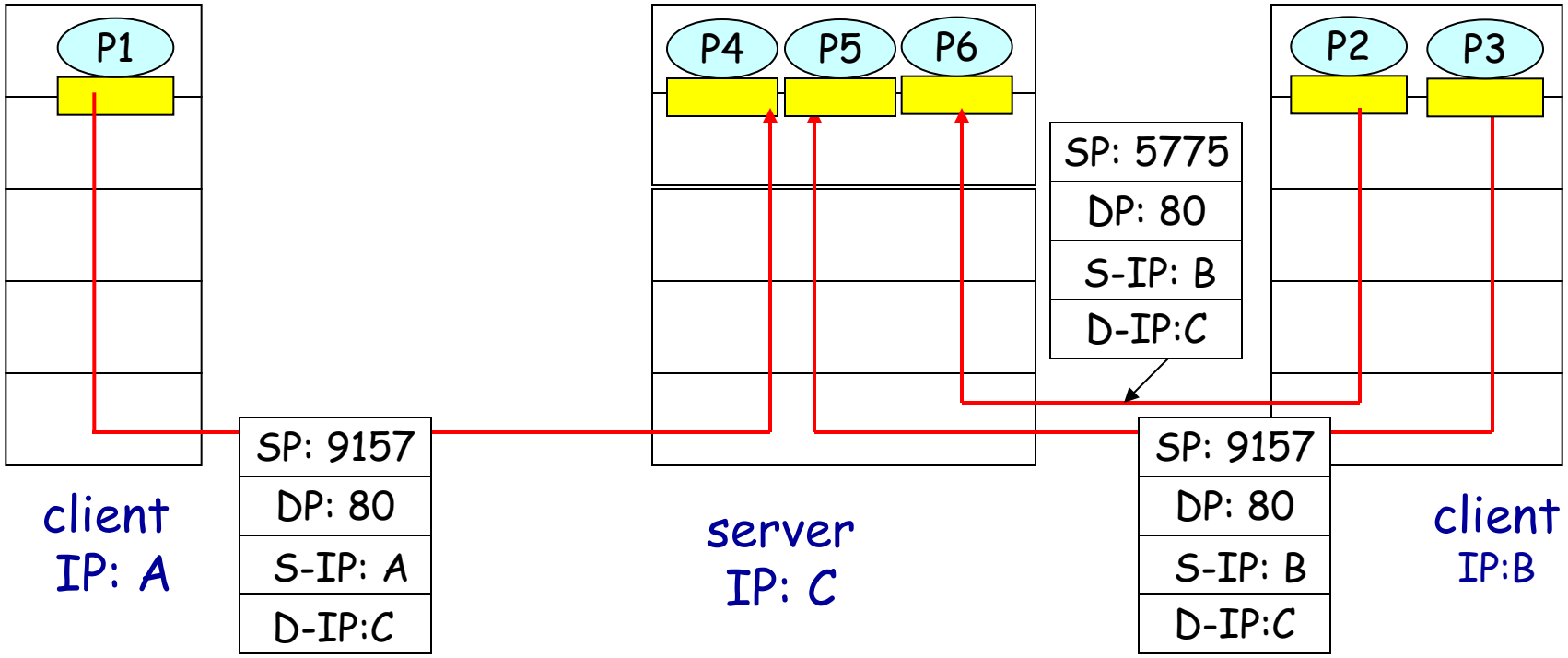


SP provides "return address"

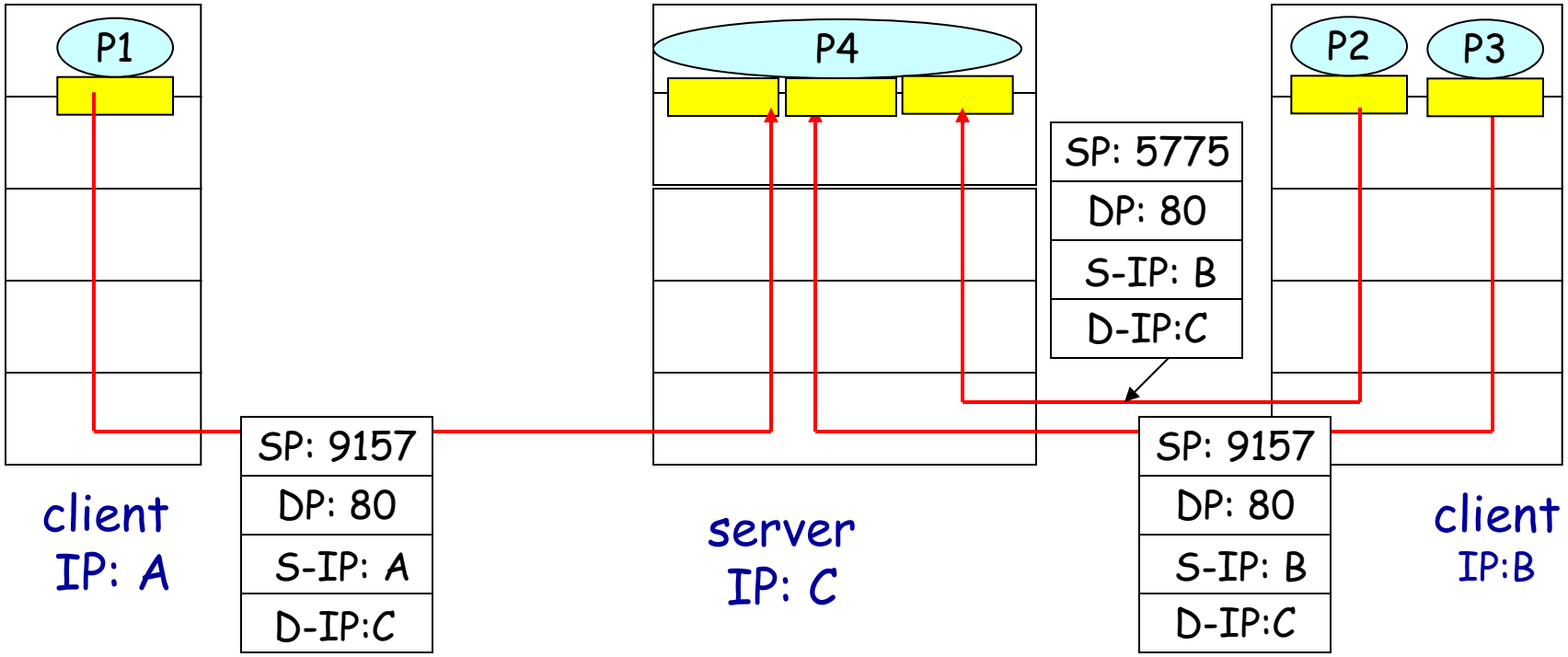
Connection-oriented demux

- TCP socket identified by 4-tuple:
  - source IP address
  - source port number
  - dest IP address
  - dest port number
- recv host uses all four values to direct segment to appropriate socket
- server host may support many simultaneous TCP sockets:
  - each socket identified by its own 4-tuple
- web servers have different sockets for each connecting client

Connection-oriented demux (cont)



Connection-oriented demux: Threaded Web Server



Chapter 5 outline

5.1 Transport-layer services

5.2 Multiplexing and demultiplexing

5.3 Connectionless transport: UDP

5.4 Connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

5.5 TCP congestion control

Applications and the underlying transport protocols

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Web	HTTP	TCP
File transfer	FTP	TCP
Remote file server	NFS	Typically UDP
Streaming multimedia	typically proprietary	UDP or TCP
Internet telephony	typically proprietary	UDP or TCP
Network management	SNMP	Typically UDP

UDP: User Datagram Protocol [RFC 768]

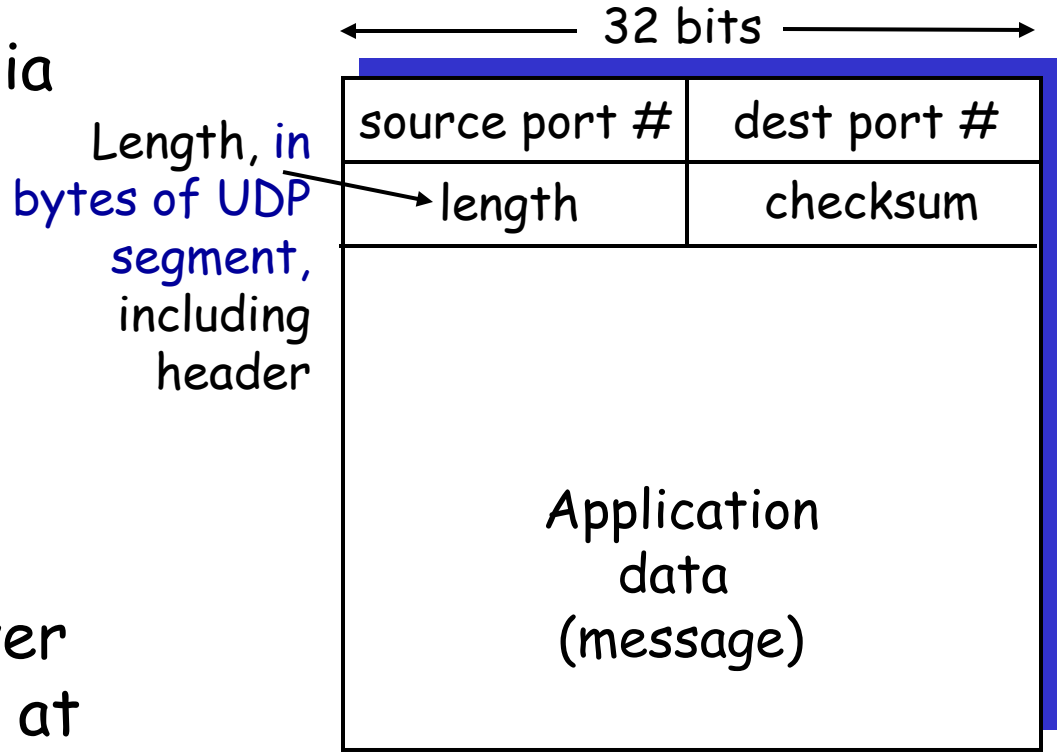
- "no frills," "bare bones" Internet transport protocol
- "best effort" service, UDP segments may be:
  - lost
  - delivered out of order to app
- *connectionless:*
  - no handshaking between UDP sender, receiver
  - each UDP segment handled independently of others

Why is there a UDP?

- no connection establishment (which can add delay)
- simple: no connection state at sender, receiver
- small segment header
- no congestion control: UDP can blast away as fast as desired

UDP: more

- often used for streaming multimedia apps
  - loss tolerant
  - rate sensitive
- other UDP uses
  - DNS
  - SNMP
- reliable transfer over UDP: add reliability at application layer
  - application-specific error recovery!



UDP segment format

UDP checksum

Goal: detect "errors" (e.g., flipped bits) in transmitted segment

Sender:

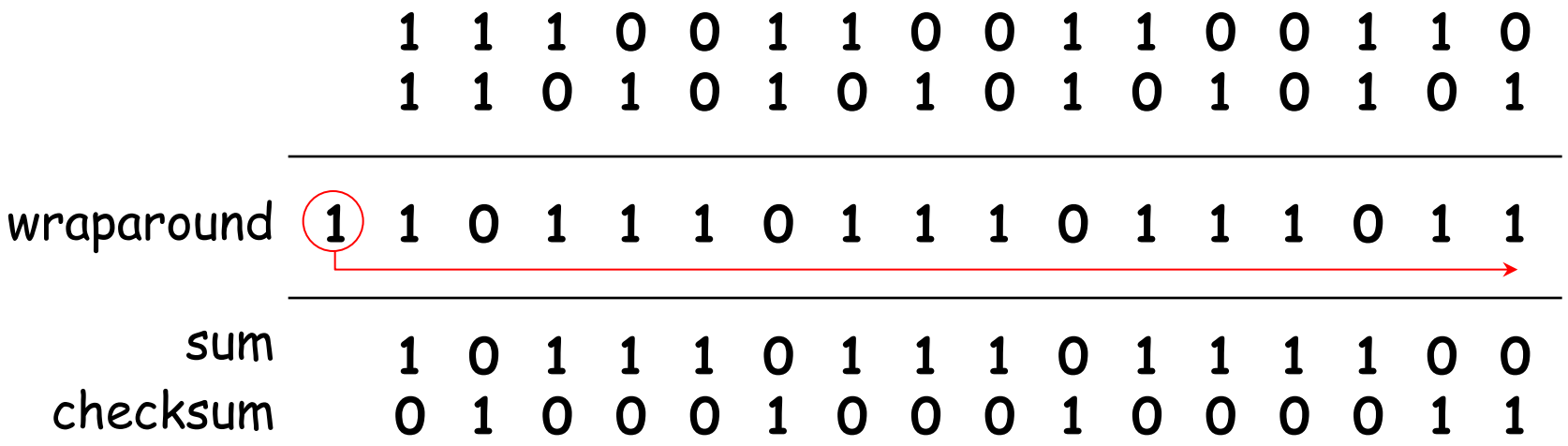
- treat segment contents as sequence of 16-bit integers
- checksum: addition (1's complement sum) of segment contents
- sender puts checksum value into UDP checksum field

Receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - NO - error detected
  - YES - no error detected

Internet Checksum Example

- Note: when adding numbers, a carryout from the most significant bit needs to be added to the result
- Example: add two 16-bit integers



Chapter 6 outline

6.1 Transport-layer services

6.2 Multiplexing and demultiplexing

6.3 Connectionless transport: UDP

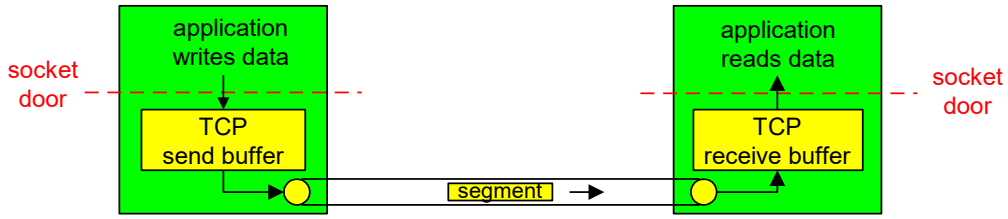
6.4 Connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

6.5 TCP congestion control

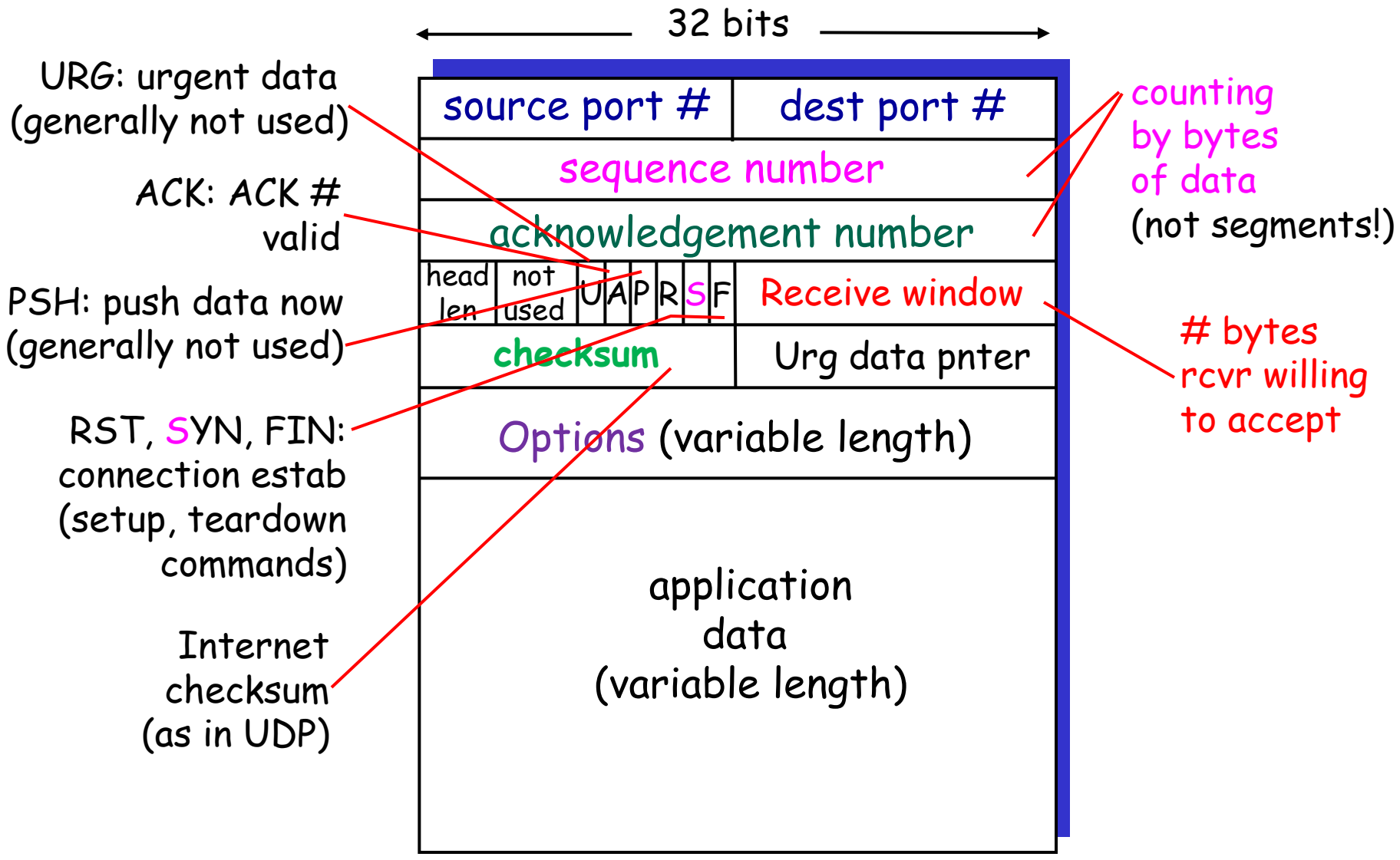
TCP: Overview RFCs: 793, 1122, 1323, 2018, 2581

- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte steam:*
- pipelined:
  - TCP congestion and flow control set window size
- *send & receive buffers*



- full duplex data:
  - bi-directional data flow in same connection
  - MSS: maximum segment size
- connection-oriented:
  - handshaking (exchange of control msgs) inits sender, receiver state before data exchange
- flow controlled:
  - sender will not overwhelm receiver

TCP segment structure



TCP seq. #'s and ACKs

Seq. #'s:

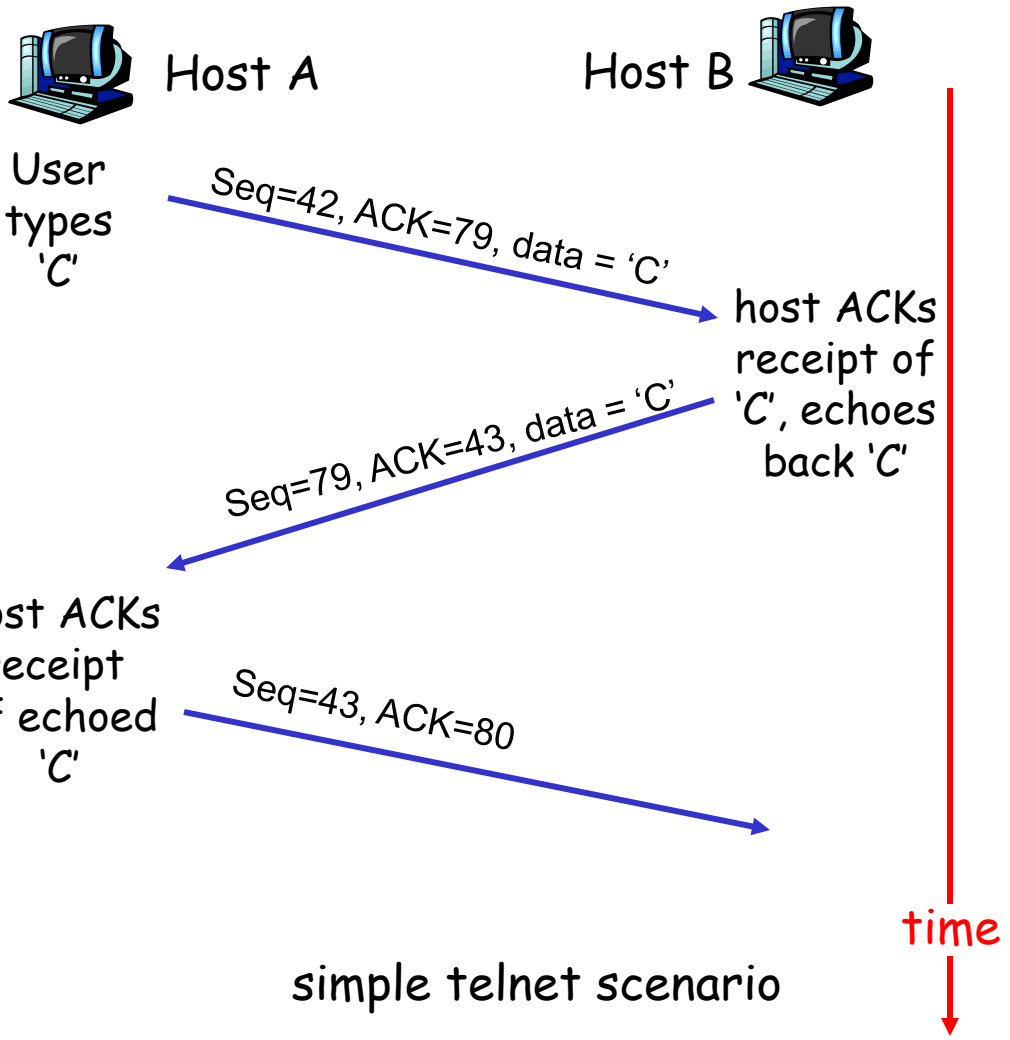
- byte stream "number" of first byte in segment's data

ACKs:

- seq # of next byte expected from other side
- cumulative ACK

Q: how receiver handles out-of-order segments

- A: TCP spec doesn't say, - up to implementor



simple telnet scenario



TCP reliable data transfer

- TCP creates reliable data transfer service on top of IP's unreliable service
- pipelined segments
- cumulative acks
- retransmissions are triggered by:
  - timeout events
  - duplicate acks

TCP sender events

data rcvd from app:

- Create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running

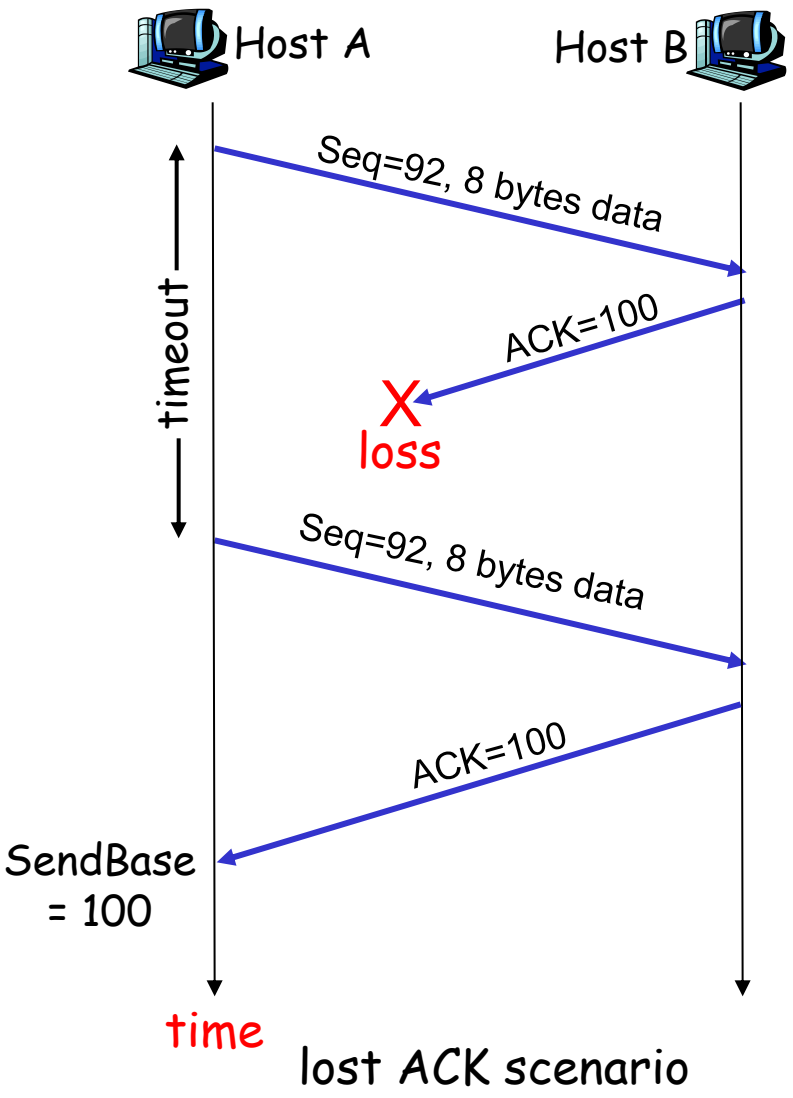
timeout:

- retransmit segment that caused timeout
- restart timer

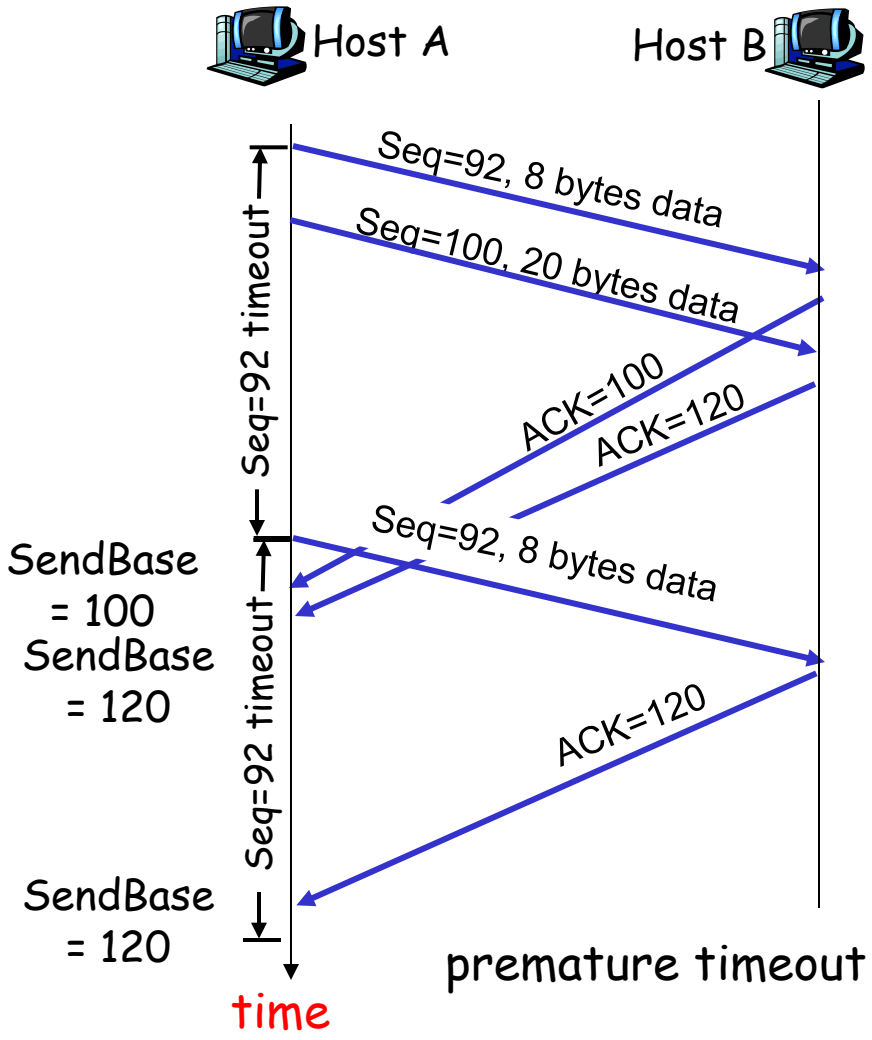
Ack rcvd:

- If acknowledges previously unacked segments
  - update what is known to be acked
  - start timer if there are outstanding segments

TCP: retransmission scenarios

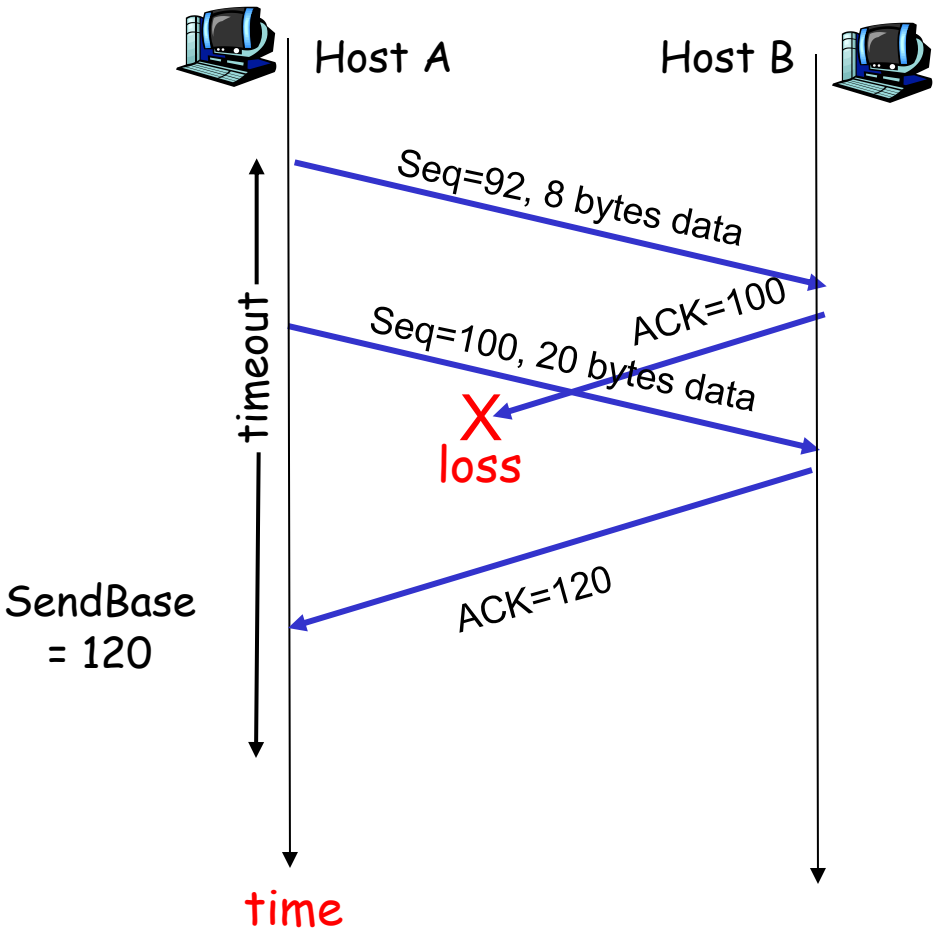


lost ACK scenario



premature timeout

TCP retransmission scenarios (more)



Cumulative ACK scenario

Fast Retransmit

- time-out period often relatively long:
  - long delay before resending lost packet
- detect lost segments via duplicate ACKs.
  - sender often sends many segments back-to-back
  - if segment is lost, there will likely be many duplicate ACKs.

- if sender receives 3 ACKs for the same data,

  fast retransmit: resend segment before timer expires

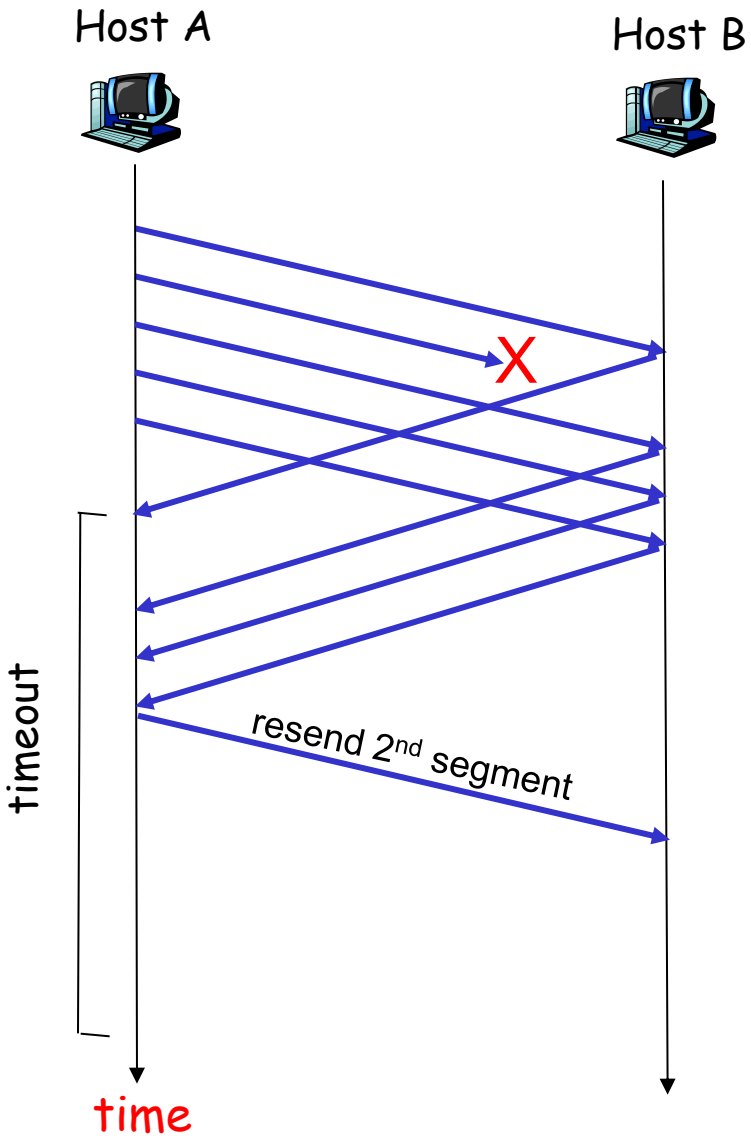
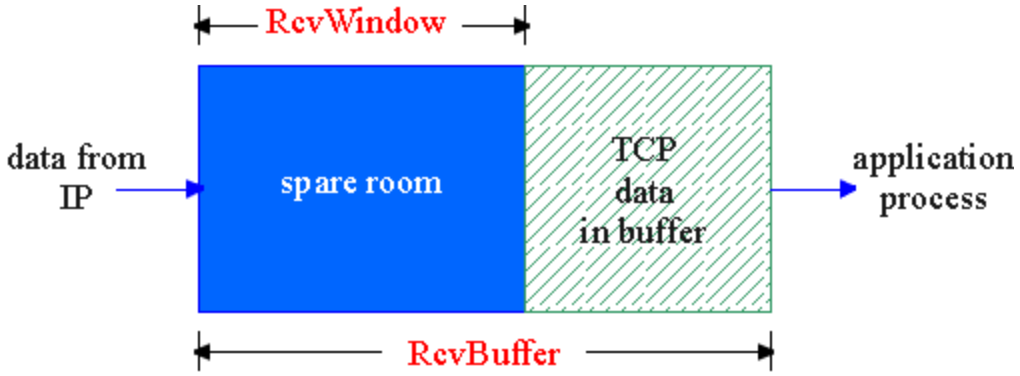


Figure 3.37 Resending a segment after triple duplicate ACK

TCP Flow Control

- receive side of TCP connection has a receive buffer:



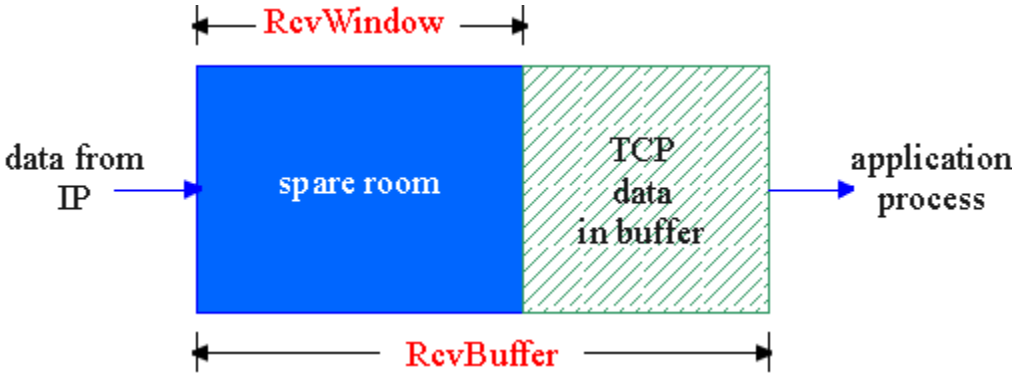
- app process may be slow at reading from buffer

flow control

sender won't overflow receiver's buffer by transmitting too much, too fast

- speed-matching service: matching the send rate to the receiving app's drain rate

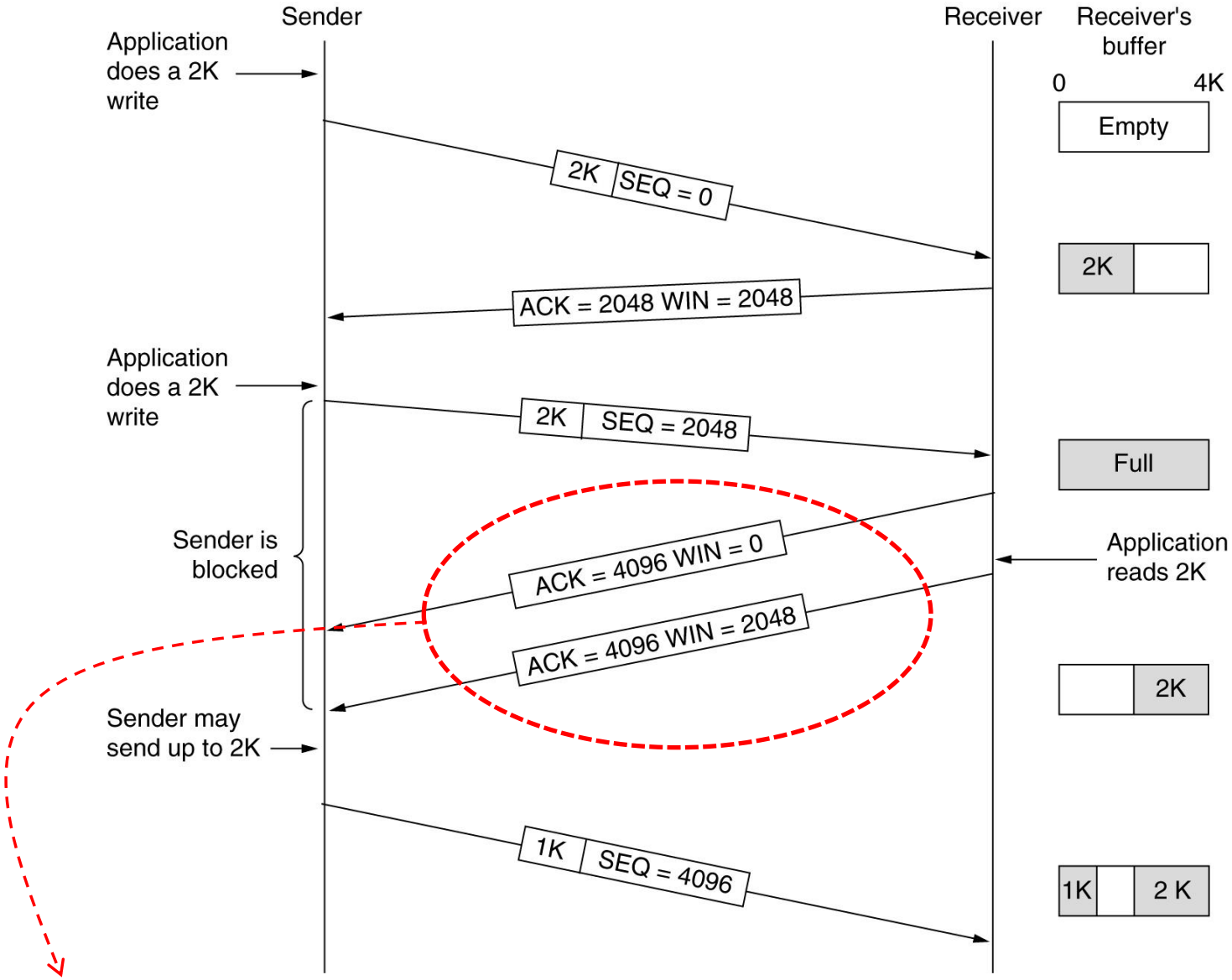
TCP Flow control: how it works



(suppose TCP receiver discards out-of-order segments)

- spare room in buffer
  = **RcvWindow**

- rcvr advertises spare room by including value of **RcvWindow** in segments
- sender limits unACKed data to **RcvWindow**
  - guarantees receive buffer doesn't overflow



- The receiver can temporarily shut down the window
- Window size = 0 → receiver asks the sender to stop transmitting data

TCP Connection Management

Recall: TCP sender, receiver establish "connection" before exchanging data segments

- initialize TCP variables:
  - seq. #s
  - buffers, flow control info (e.g. **RcvWindow**)
- *client:* connection initiator

```
Socket clientSocket = new
Socket("hostname","port
number");
```

- *server:* contacted by client

```
Socket connectionSocket =
welcomeSocket.accept();
```

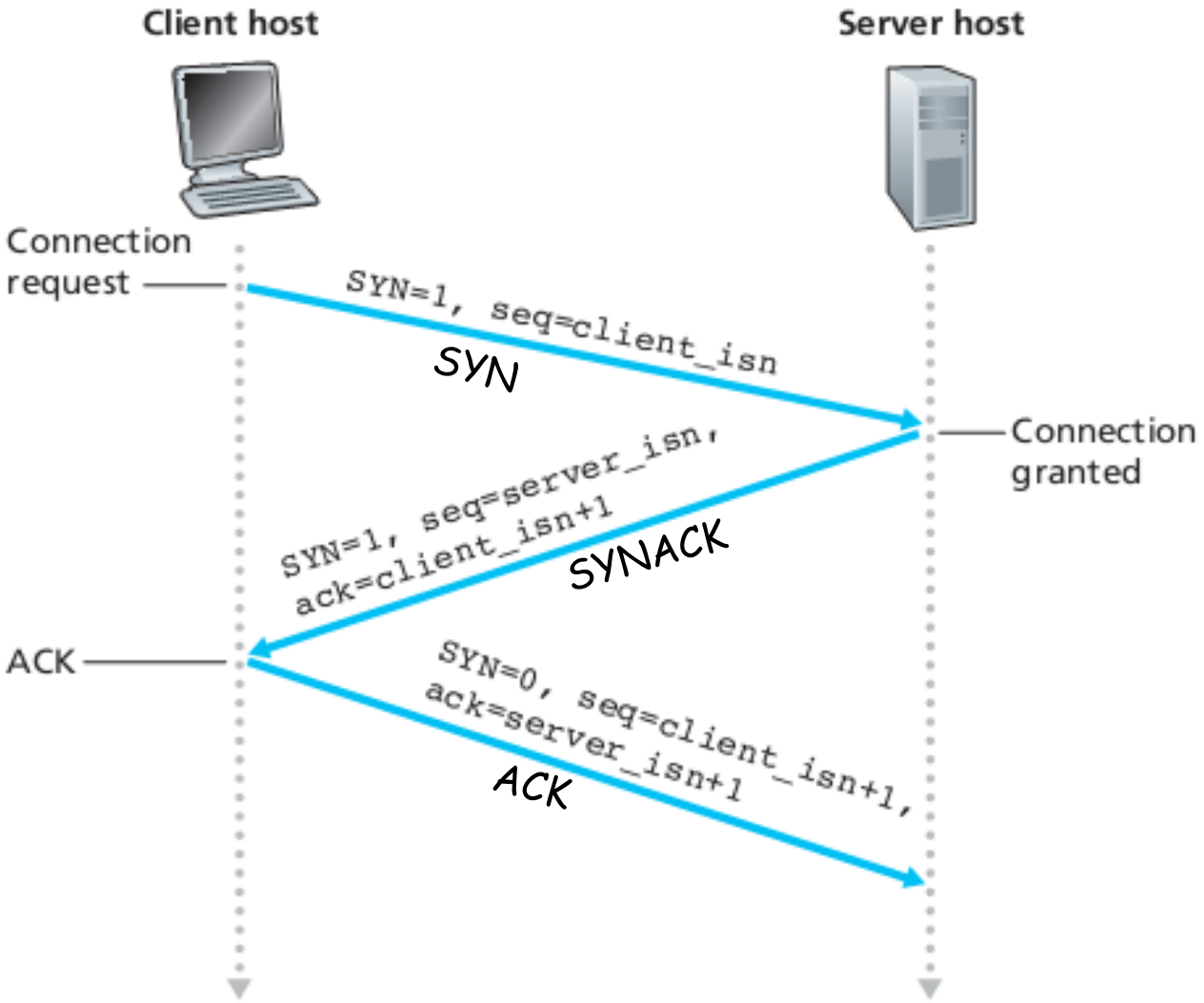
Three way handshake:

Step 1: client host sends TCP SYN segment to server
- specifies initial seq #
- no data

Step 2: server host receives SYN, replies with SYNACK segment
- server allocates buffers
- specifies server initial seq. #

Step 3: client receives SYNACK, replies with ACK segment, which may contain data

Three-way handshake



SYN Flood Attack

Denial of Service (DoS) attack known as the **SYN flood attack**.

The attacker(s) send a large number of TCP SYN segments, without completing the third handshake step. With this deluge of SYN segments, the server's connection resources become exhausted as they are allocated; legitimate clients are then denied service.

The attack can be amplified by sending the SYNs from multiple sources, creating a DDoS (Distributed Denial of Service) SYN flood attack

SYN Flood Attack

- An effective defense known as **SYN cookies** [RFC 4987]
- When receiving a SYN segment, the server, instead of allocating resources for this SYN, it creates an TCP sequence number ("cookie")

  TCP sequence number= hash (IP addrs of S and D, port numbers of SYN segment, and a secret number only known to the server)

- A legitimate client will return an ACK segment. When the server receives this ACK, it must verify that the ACK corresponds to some SYN sent earlier. But how is this done if the server maintains no memory about SYN segments?
- Run the same hash function using same fields in ACK segment and the secret key. If the result of the function plus one is the same as the acknowledgment, the server concludes that the ACK corresponds to an earlier SYN segment and is hence valid. The server then allocate resources.

TCP Connection Management (cont.)

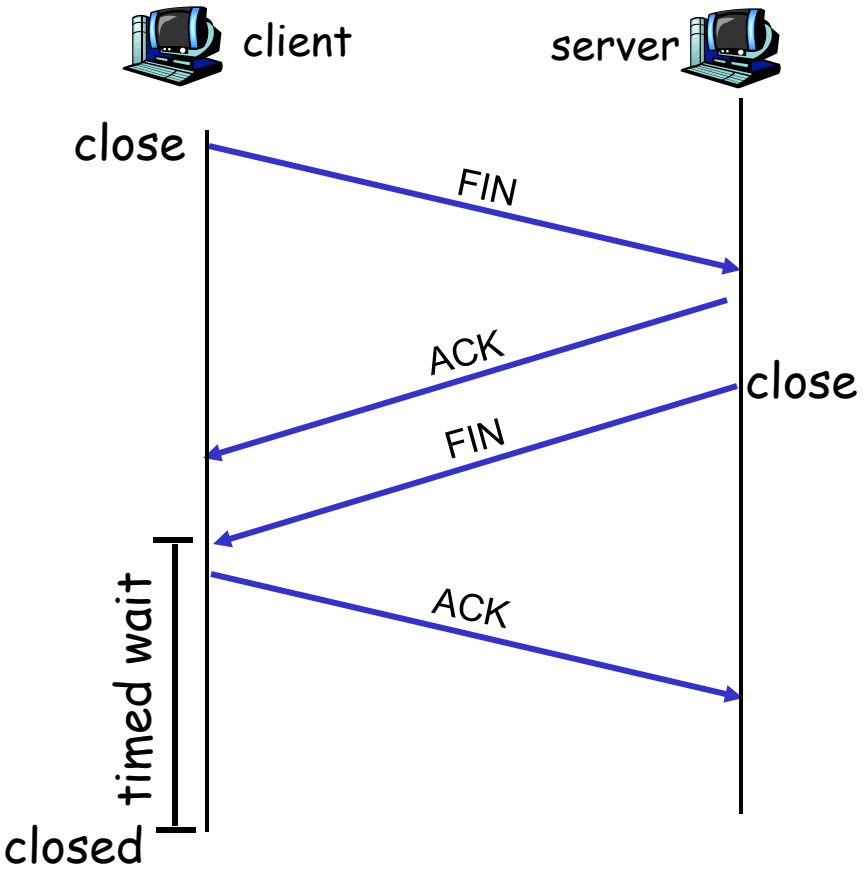
Closing a connection:

client closes socket:

```
clientSocket.close();
```

Step 1: client end system sends TCP FIN control segment to server

Step 2: server receives FIN, replies with ACK. Closes connection, sends FIN.

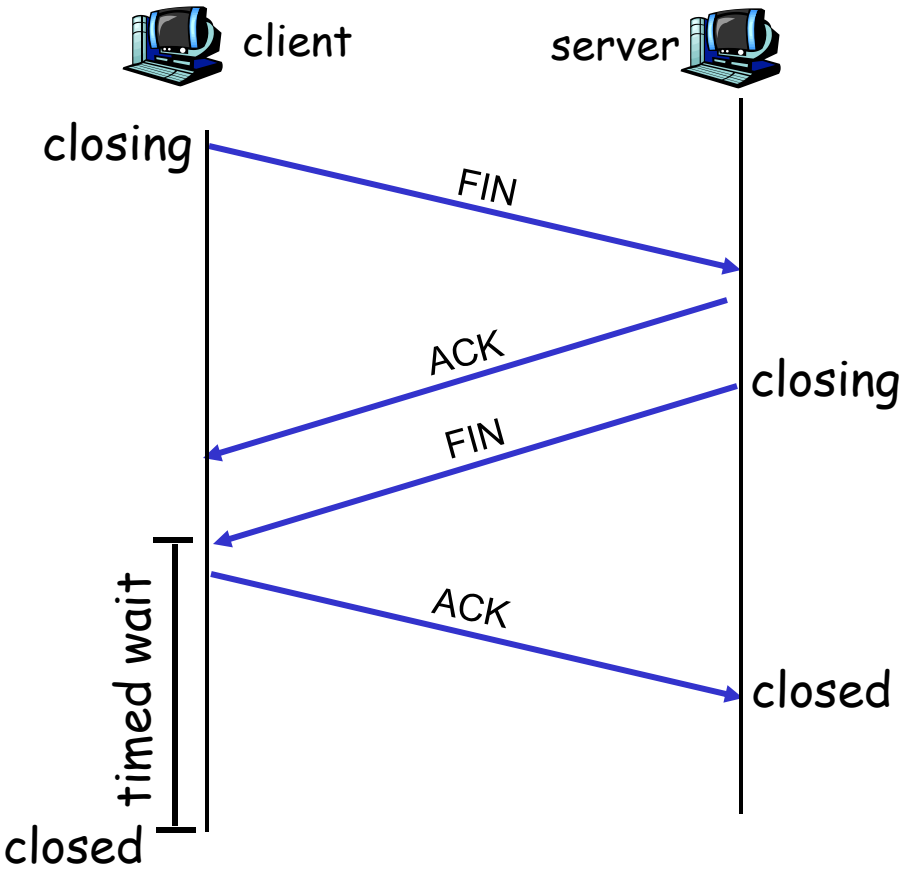


TCP Connection Management (cont.)

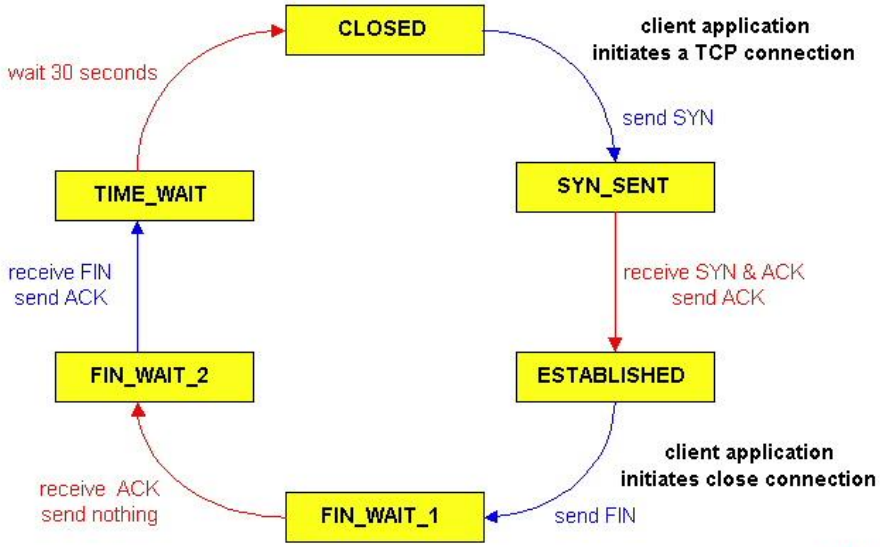
Step 3: client receives FIN, replies with ACK.

- Enters "timed wait" - will respond with ACK to received FINs

Step 4: server, receives ACK. Connection closed.

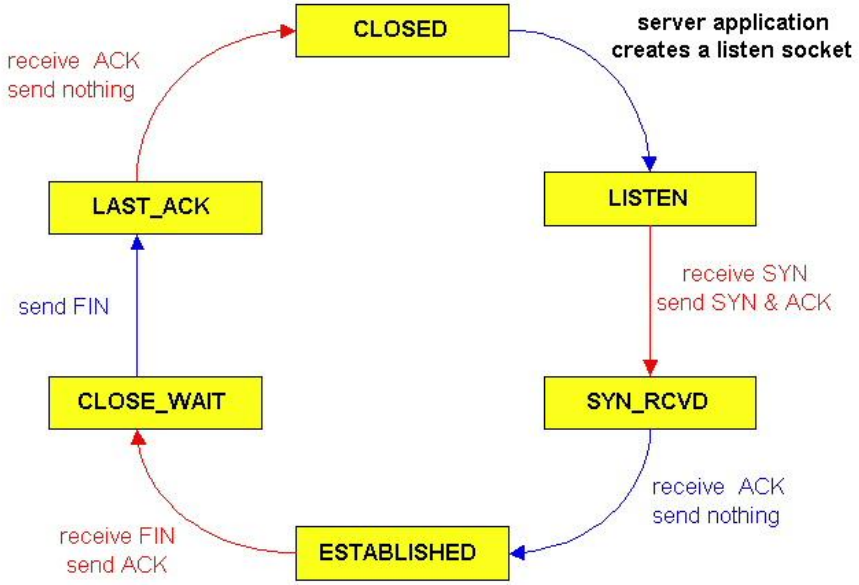


TCP Connection Management (cont)



TCP client lifecycle

TCP server lifecycle



Chapter 5 outline

5.1 Transport-layer services

5.2 Multiplexing and demultiplexing

5.3 Connectionless transport: UDP

5.4 Connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

5.5 TCP congestion control

Congestion Control

Congestion:

- informally: "too many sources sending too much data too fast for *network* to handle"
- Congestion occurs when total arrival rate from all flows exceeds a router's output link capacity
- different from flow control!
- manifestations:
  - lost packets (buffer overflow at routers)
  - long delays (queueing in router buffers)
- To alleviate congestion, the sending nodes should slow down the rate of transmission.
- How? by reducing the window. The window size is the number of bytes that are allowed to be in flight simultaneously

Congestion Control

Two broad approaches:

1- End-end congestion control: no explicit feedback from network - congestion inferred from end-system observed loss, delay - approach taken by TCP

2- Network-assisted congestion control: routers provide feedback to end systems indicating congestion, used in Asynchronous Transfer Mode (ATM)

Congestion Control

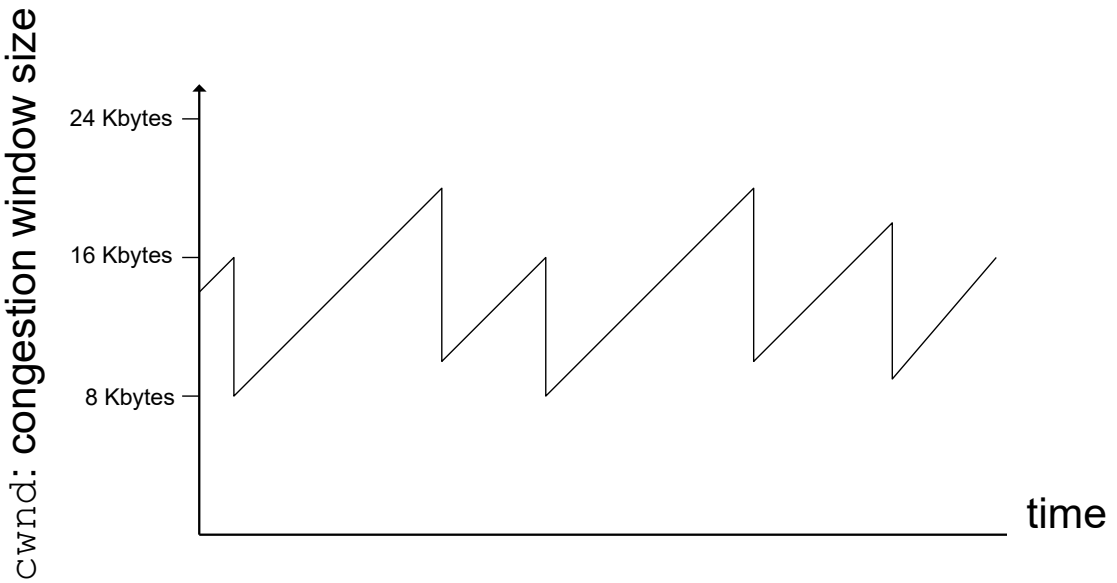
- Receiver window (rwnd) is used to ensure that receiver's buffer will not overflow (flow control)
- Congestion window (cwnd) is used to ensure that the sender will not overflow the intermediate routers' buffers between source and destination (congestion control)
- The actual window size = min(rwnd, cwnd) to avoid overflowing the receiver or the routers
- That means max-in-flight bytes ≤ min(rwnd, cwnd)

TCP Congestion Control

Additive increase, multiplicative decrease (AIMD)

- *approach:* increase transmission rate (window size), probing for usable bandwidth, until loss occurs
  - *additive increase:* increase **cwnd** by 1 maximum segment size (MSS) every RTT until loss detected
  - *multiplicative decrease:* cut **cwnd** in half after loss

saw tooth behavior: probing for bandwidth



TCP Congestion Control: details

- sender limits transmission:

  LastByteSent-LastByteAcked

  ≤ cwnd

- roughly,

$$\text{rate} = \frac{\text{cwnd}}{\text{RTT}} \text{ Bytes/sec}$$

- **cwnd** is dynamic, function of perceived network congestion

How does sender perceive congestion?

- loss event = timeout *or* 3 duplicate acks
- TCP sender reduces rate (**cwnd**) after loss event

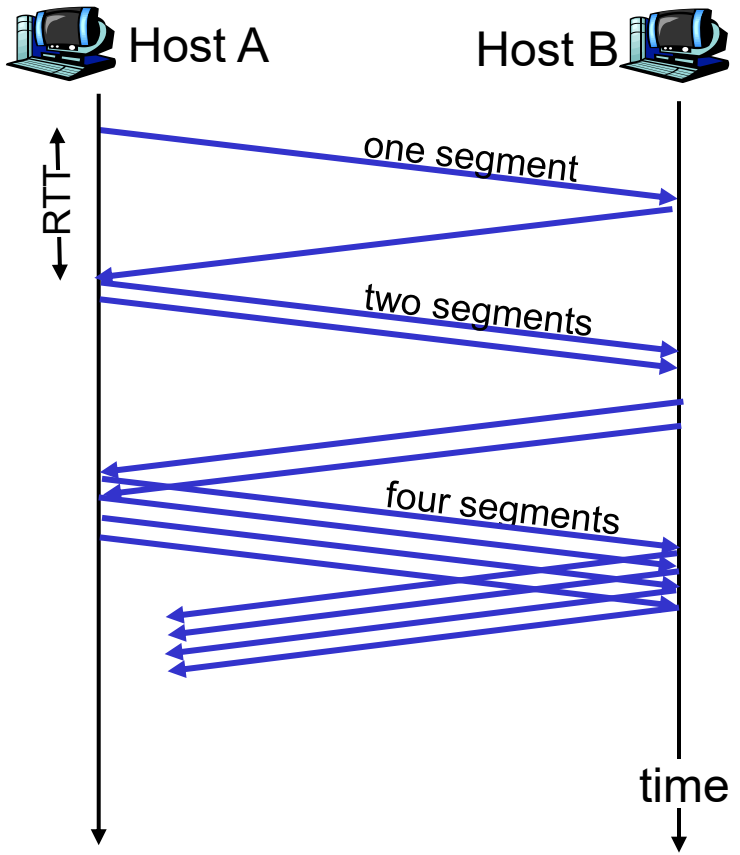
three mechanisms:

- AIMD
- slow start
- conservative after timeout events

1. TCP Slow Start

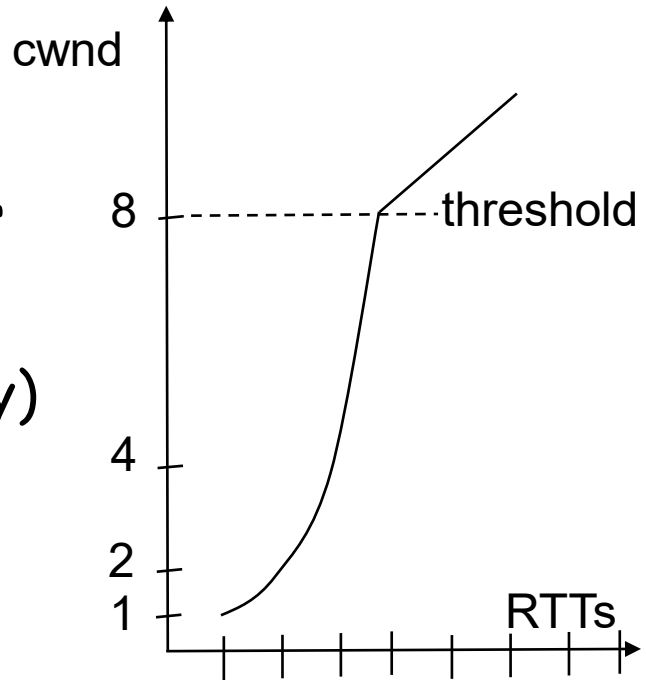
- when connection begins, increase rate exponentially until first loss event:
  - initially **cwnd** = 1 MSS
  - double **cwnd** every RTT
  - done by incrementing **cwnd** for every ACK received

- Discover available bandwidth fast - desirable to quickly ramp up to respectable rate
  - When cwnd > threshold, move to congestion avoidance phase to slow down the sending rate



2. Congestion avoidance phase

- When Cwnd is above a threshold
- Cwnd is incremented by one segment every RTT (= for every window of ACKs it receives) → linear increase
- Cwnd continues to increase (linearly) until loss is detected
- TCP spends most of its time in this phase



3. Reaction to congestion phase

- At any time, when congestion occurs, decrease the window size.
- How TCP recognizes congestion? Two congestion indication mechanisms

1. Three duplicated ACKs:

Duplicate ACKs means the receiver got all packets up to the gap and is actually receiving packets → at least network capable of delivering some segments. Could be due to temporary congestion

- Reduce Cwnd but not aggressively
- Congestion threshold = cwnd/2 and new cwnd = threshold
- Stay in congestion avoidance phase

3. Reaction to congestion phase

2. Timeout

No response from receiver - more likely due to significant congestion → reduce cwnd aggressively

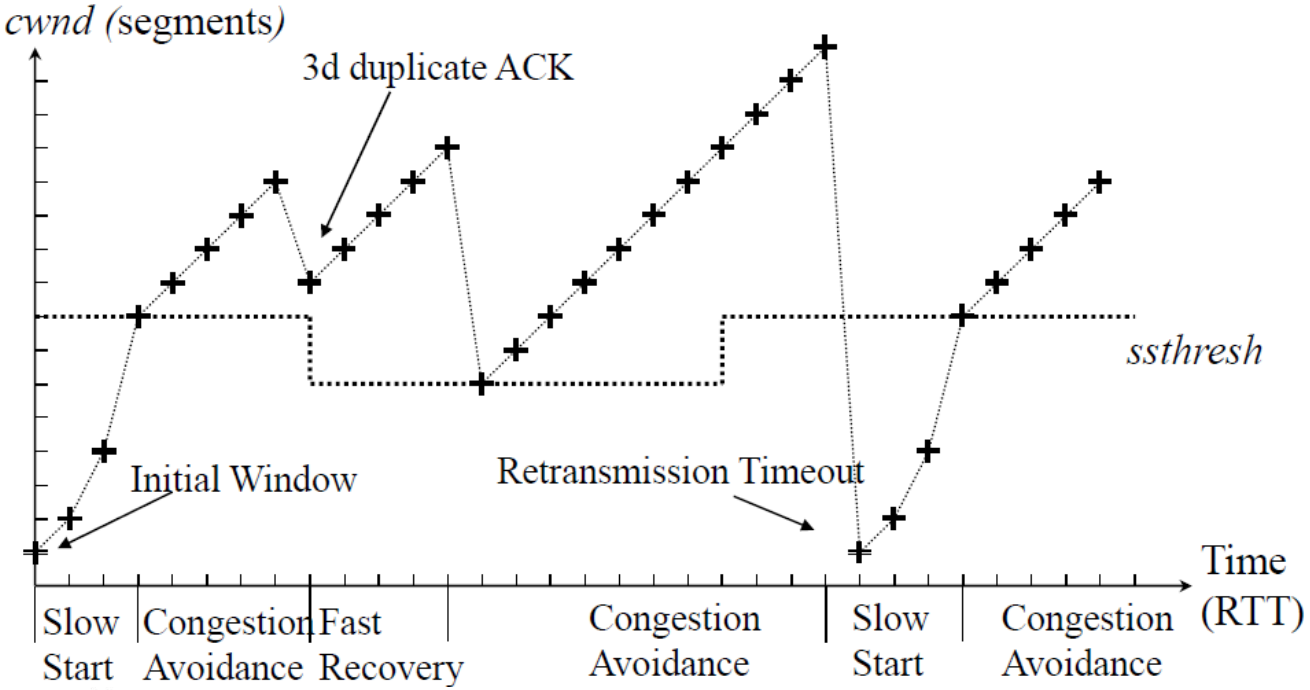
- Congestion threshold = cwnd/2 and new cwnd = 1 max. segment size (MSS)

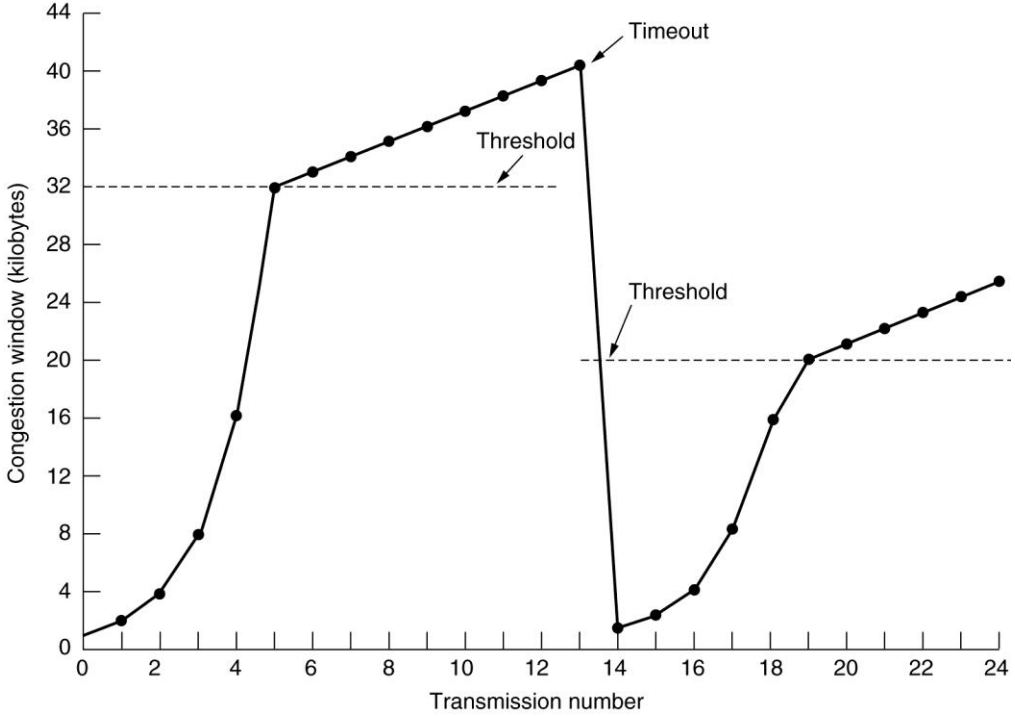
- Go back to slow start phase

- Most of the time the window will be like a sawtooth

  Additive Increase/Multiplicative Decrease (AIMD): cwnd increases by 1 every RTT, cwnd decreases by a factor of two with every loss, and repeat

Examples


cwnd (segments)
3d duplicate ACK
ssthresh
Initial Window
Retransmission Timeout
Time (RTT)
Slow Start
Congestion Avoidance
Fast Recovery
Congestion Avoidance
Slow Start
Congestion Avoidance


Timeout
Threshold
Threshold
Congestion window (kilobytes)
Transmission number
0
4
8
12
16
20
24
28
32
36
40
44
0
2
4
6
8
10
12
14
16
18
20
22
24

Chapter 5: Summary

- principles behind transport layer services:
  - multiplexing, demultiplexing
  - flow control
  - congestion control
- instantiation and implementation in the Internet
  - UDP
  - TCP